

Quiz 03 — Arreglos dinámicos y análisis amortizado

Estructuras de Datos 2026-2 · analysis.pdf

1. Representación interna

DynamicArray<T>. Tres campos: `T[] data` (almacenamiento), `size` (elementos vivos) y `capacity = data.length`. Los elementos viven en `data[0..size-1]`; `append` escribe en `data[size]` y, si `size == capacity`, primero llama a `resize()` (nuevo arreglo de capacidad $2 \cdot \text{capacity}$ y copia de los `size` elementos).

Snake con buffer circular. Convención del enunciado: el índice lógico 0 es la **cola** y el índice `size-1` es la **cabeza**. Para no desplazar los `k` elementos en cada movimiento, el cuerpo se guarda en un arreglo circular con campos `array`, `capacity`, `head` (índice físico de la cabeza) y `size` (longitud). `addHead` hace retroceder `head` una celda ($\text{mod } \text{capacity}$), así que la serpiente crece hacia índices físicos menores y la cola queda al otro extremo. El índice lógico i ($0 \leq i < \text{size}$) se ubica en $\text{physical}(i) = (\text{head} + \text{size} - 1 - i) \% \text{capacity}$. La cola es $\text{physical}(0) = (\text{head} + \text{size} - 1) \% \text{capacity}$ (no se guarda: se deriva). El argumento de `%` nunca es negativo porque $i \leq \text{size}-1$, y por eso el resultado siempre está en $[0, \text{capacity}-1]$.

```
void addHead(Position p) {
    if (size == capacity) resize();
    head = (head - 1 + capacity) % capacity;
    array[head] = p; size++;
}

Position removeTail() {
    if (size == 0) throw new IllegalStateException("Snake vacía");
    Position t = array[(head + size - 1) % capacity];
    size--; // la celda liberada queda como basura (I5)
    return t;
}

Position get(int i) {
    if (i < 0 || i >= size) throw new IndexOutOfBoundsException();
    return array[(head + size - 1 - i) % capacity];
}

private void resize() {
    Position[] n = new Position[2 * capacity];
    for (int j = 0; j < size; j++) n[j] = array[(head + j) % capacity]; // cabeza primero
    array = n; capacity *= 2; head = 0;
}
```

Un movimiento ordinario es `addHead` seguido de `removeTail` (dos operaciones $O(1)$ salvo el `resize`); si la serpiente come, solo se ejecuta `addHead`. El `resize` pone `head = 0` pero ocurre solo al llenarse el arreglo, no en un movimiento ordinario, así que no viola la restricción de no desplazar sistemáticamente los `k` elementos.

2. Invariantes

DynamicArray: (D1) $\text{capacity} \geq 1$ (se valida $\text{initialCapacity} \geq 1$); (D2) $0 \leq \text{size} \leq \text{capacity}$ (se preserva en `append`, `resize` y `removeLast`); (D3) los índices válidos son $[0, \text{size})$: `get/set` lanzan `IndexOutOfBoundsException` fuera de ese rango.

Snake circular:

#	Enunciado	Por qué se conserva
I1	$\text{capacity} = \text{array.length} \geq 2$ y es potencia de 2	Se inicia en 2 y <code>resize</code> solo duplica.
I2	$0 \leq \text{size} \leq \text{capacity}$	<code>addHead</code> solo inserta tras garantizar $\text{size} < \text{capacity}$ (<code>resize</code> si está llena); <code>removeTail</code> solo decrementa con $\text{size} > 0$.
I3	$0 \leq \text{head} < \text{capacity}$ y, si $\text{size} > 0$, <code>array[head]</code> es la cabeza (índice lógico $\text{size}-1$) y <code>array[(head+size-1)%capacity]</code> es la cola (índice 0)	El módulo mantiene <code>head</code> en rango; <code>addHead</code> escribe justo en el nuevo <code>head</code> ; <code>removeTail</code> solo acorta por el lado de la cola.
I4	$\text{physical}(i) = (\text{head} + \text{size} - 1 - i) \% \text{capacity}$ es válida (en $[0, \text{capacity}-1]$) e inyectiva sobre $0 \leq i < \text{size}$	Argumento $\geq \text{head} \geq 0$ porque $i \leq \text{size}-1$, y <code>%</code> lo deja en rango. Si $\text{physical}(i) = \text{physical}(j)$, entonces $i \equiv j \text{ mod } \text{capacity}$; como $ i-j < \text{size} \leq \text{capacity}$, $i = j$.
I5	Toda celda que no sea <code>physical(i)</code> con $0 \leq i < \text{size}$ es basura y no forma parte del estado	<code>removeTail</code> solo baja <code>size</code> ; <code>addHead</code> sobrescribe basura o una celda libre, nunca un elemento vivo (por I2).
I6	<code>resize</code> conserva el orden lógico	Copia <code>n[j] = array[(head+j)%capacity]</code> para $j = 0..\text{size}-1$ (cabeza primero) y reinicia <code>head = 0</code> ; entonces $\text{physical}(i) = \text{size}-1-i$ sigue apuntando al mismo elemento.

Corrección de las operaciones. `get(i)` es correcta por I3 e I4; `addHead` deja el nuevo elemento como índice lógico $\text{size}-1$ (la nueva cabeza) sin mover físicamente a los demás; `removeTail` devuelve el índice lógico 0 y, al bajar `size`, la celda liberada deja de importar por I5.

Traza (estado inicial $\text{size} = \text{capacity} = 2$, cola A, cabeza B; N1 no come, N2 come, N3 no come, N4 no come, N5 come, N6 no come). “.” = celda vacía, [x] = celda basura (fuera del estado lógico).

Paso	¿Resize?	capacity / head / size	Arreglo físico (0..cap-1)	Cuerpo lógico (cola→cabeza)
Inicio	—	2 / 1 / 2	a b	a, b
N1 (no come)	Sí (2→4)	4 / 3 / 2	b [a] · c	b, c
N2 (come)	No	4 / 2 / 3	b [a] d c	b, c, d
N3 (no come)	No	4 / 1 / 3	[b] e d c	c, d, e
N4 (no come)	No	4 / 0 / 3	f e d [c]	d, e, f
N5 (come)	No	4 / 3 / 4	f e d g	d, e, f, g
N6 (no come)	Sí (4→8)	8 / 7 / 4	g f e [d] · · · h	e, f, g, h

Cada fila muestra el estado después del movimiento completo (addHead y, si no come, removeTail). En N1 el resize copia [a, b] como [b, a] con head = 0, luego head = 3 recibe c y removeTail saca a. En N6 el resize copia [g, f, e, d], luego head = 7 recibe h y removeTail saca d.

3. Análisis de complejidad

Operación	Peor caso	Justificación
get(i), set(i, v)	$O(1)$	Acceso directo por índice; no se recorre el arreglo.
removeLast()	$O(1)$	Solo decrementa size.
append(x)	$O(n)$ peor caso · $O(1)$ amortizado	Si size = capacity, resize copia n elementos; ocurre solo al llenarse (ver §4).
Snake con arreglo plano: removeTail	$O(k)$	Desplaza $k-1$ elementos una posición ($i-1 \leftarrow i$); size-1 es $O(1)$.
Snake con arreglo plano: MOVE	$O(k)$	append ($O(1)$ amort.) + removeTail ($O(k)$). Con k constante, m movimientos cuestan $O(mk) = O(m)$.
Circular: addHead / removeTail / get	$O(1)$ (addHead: amortizado)	Aritmética modular sin desplazar elementos; solo addHead puede disparar resize.

¿Que append sea $O(1)$ amortizado implica que MOVE también lo sea? Falso con el arreglo plano. MOVE = append + removeTail. Cada removeTail hace exactamente $k-1$ desplazamientos (no solo en el peor caso), así que ningún argumento de amortización puede esconder ese costo: m movimientos con longitud k cuestan al menos $m(k-1) = \Theta(mk)$, es decir $\Theta(k)$ por MOVE, que no es $O(1)$ cuando k crece. El **cuello de botella es removeTail**, que desplaza los elementos uno por uno. El buffer circular elimina ese desplazamiento.

4. Demostración del análisis amortizado

Que el peor caso sea $O(n)$ no contradice el $O(1)$ amortizado: el peor caso describe una operación aislada; el amortizado, el promedio sobre una secuencia. El resize no ocurre en cada append, sino solo cuando el arreglo se llena, y el costo se reparte entre las operaciones anteriores.

(a-b) Método agregado. Sea g la cantidad de manzanas comidas y m el número de movimientos ($g \leq m$). La longitud inicial es 2 y, como addHead ocurre antes de removeTail, la longitud máxima transitoria es $n = g + 3$. Un resize ocurre solo cuando size = capacity = C, y copia exactamente C elementos. Las capacidades en esos instantes son 2, 4, ..., 2^k , con $2^k < n$ (el último resize se disparó para insertar el elemento $2^k + 1 \leq n$). Por lo tanto: copias = $2 + 4 + \dots + 2^k = 2^{k+1} - 2 < 2n - 2$. Las copias totales son $O(n) = O(g)$, no cuadráticas.

(c) Costo total y amortizado. Con el modelo de costos del enunciado: se escribe una posición nueva por movimiento (m en total), las copias suman menos de $2n-2 = 2g+4$, y las operaciones aritméticas son $O(1)$ por movimiento. Costo total $\leq m + (2g+4) + O(m) = O(m + g) = O(m)$, porque $g \leq m$. El costo amortizado de addHead es $O(m)/m = O(1)$.

(d) Papel de m y g. m fija el costo base (cuántas veces se ejecutan las instrucciones de tiempo constante). g gobierna el crecimiento real en memoria: es lo único que puede llenar el arreglo y provocar resizes, así que el número de copias depende de g y no de m.

(e) Método del banquero. Cobramos 3 créditos por cada addHead: 1 paga la inserción; 1 se deposita sobre el elemento nuevo (para su futura copia); 1 se deposita sobre un elemento antiguo vivo que aún no tenga crédito. Tras un resize a capacidad C quedan a lo sumo $C/2 + 1$ elementos vivos, todos copiados (los antiguos). Para volver a llenarse hace falta que haya C elementos vivos, así que al menos $C/2$ de ellos se insertaron después del resize y traen su propio crédito, y en ese lapso hubo al menos $C/2$ addHead que recargaron a los $\leq C/2$ elementos antiguos que siguen vivos. Así cada uno de los C elementos tiene 1 crédito y el resize (que cuesta C) queda pagado. **removeTail:** elimina el elemento más antiguo; si tenía crédito, se pierde, pero ese elemento nunca volverá a copiarse, por lo que su crédito ya no se necesita. Los créditos de los elementos vivos no cambian, luego la cuenta sigue siendo suficiente y removeTail no rompe el análisis. El costo amortizado de addHead es $\leq 3 = O(1)$.

5. Comparación de políticas de crecimiento

Se parte de capacidad 2 y se insertan n elementos, con n operaciones de crecimiento; costo amortizado = copias totales / n.

(a) La elección de cuándo crece la serpiente no cambia las copias. Sin contracción, la capacidad solo aumenta y solo cambia en un resize, que ocurre únicamente con el arreglo lleno y copia exactamente capacity elementos. La

secuencia de capacidades $c_0 = 2$, $c_1 = f(c_0)$, ... la fija la política, y los resizes que ocurren hasta llegar a longitud n son exactamente los que tienen $c_i < n$, cada uno con costo c_i . El adversario solo decide en qué momento se llena el arreglo, no cuántos resizes hacen falta ni cuánto cuesta cada uno. Para aumentar las copias tendría que controlar el resize (forzar uno con el arreglo no lleno, o cambiar la política) o poder *contraer* el arreglo para hacerlo crecer y copiar otra vez.

A: C+1. Capacidades $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \dots$: hay $n-2$ resizes, $O(n)$. Cada resize copia toda la capacidad actual: $2+3+\dots+(n-1) = O(n^2)$ copias; amortizado $O(n)$.

B: 2C. Capacidades $2 \cdot 2^r$: se necesitan $2^{r+1} \geq n$, o sea $r+1 \geq \log_2 n \rightarrow O(\log n)$ resizes; copias $2+4+\dots+2^r < 2n \rightarrow O(n)$; amortizado $O(1)$.

C: [1.5C]. Capacidades $2 \rightarrow 3 \rightarrow 5 \rightarrow 8 \dots$, aprox. $2 \cdot 1.5^r \geq n \rightarrow r \geq \log_{1.5}(n/2) \rightarrow O(\log n)$ resizes; copias: serie geométrica de razón 1.5, $O(n)$; amortizado $O(1)$.

Espacio sin usar justo después de un resize (C = capacidad previa, ya insertado el nuevo elemento): A: $(C+1) - (C+1) = 0$; B: $2C - (C+1) = C - 1$ ($\approx$ mitad de la nueva capacidad); C: $1.5C - (C+1) = 0.5C - 1$ ($\approx$ un tercio de la nueva capacidad).

Política	# Resizes	Total de copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
$C + 1$	$O(n)$	$O(n^2)$	$O(n)$	0
$2C$	$O(\log_2 n)$	$O(n)$	$O(1)$	$C - 1$ ($\approx 1/2$ de la capacidad nueva)
$[1.5C]$	$O(\log_{1.5} n)$	$O(n)$	$O(1)$	$0.5C - 1$ ($\approx 1/3$ de la capacidad nueva)

Conclusión. Ninguna política domina. A minimiza memoria pero destruye el tiempo (copias cuadráticas). B minimiza resizes y copias pero puede desperdiciar hasta la mitad del arreglo. C cambia un poco más de resizes y copias por menos desperdicio.

¿Hace falta duplicar ($\alpha = 2$)? No. Cualquier política multiplicativa $C_{\text{new}} = \lceil \alpha C \rceil$ con $\alpha > 1$ constante da copias $O(n)$ y amortizado $O(1)$. Como $\lceil \alpha C \rceil \geq \alpha C$, cada capacidad es al menos α veces la anterior; si $c_1 < c_2 < \dots < c_r < n$ son las capacidades en los resizes, las copias suman $\sum c_i \leq c_r(1 + 1/\alpha + 1/\alpha^2 + \dots) = c_r \cdot \alpha/(\alpha-1) < \alpha n/(\alpha-1) = O(n)$. Dividiendo entre las n operaciones queda $O(1)$. Por tanto copias $< \alpha n/(\alpha-1)$ para todo $\alpha > 1$ (la constante $\alpha/(\alpha-1)$ crece sin límite cuando $\alpha \rightarrow 1$). El $\lceil \cdot \rceil$ garantiza crecimiento estricto: con $[1.25C]$ y $C = 2$ la capacidad no cambiaría ($[2.5] = 2$), mientras que $[2.5] = 3$. El factor $\alpha/(\alpha-1)$ muestra el trade-off: α cercano a 1 se parece a A (muchas copias, poco desperdicio, $\approx (\alpha-1)/\alpha$ de la capacidad libre) y α grande se parece a B.